Bahria University Lahore Campus

Department of Computer Science

Artificial Intelligence – BSCS 5

Assignment 2

A Genetic-Algorithm Strategy Engine for UAV Fleet Deployment, Sensor Scheduling and Path Planning over a Volatile Hazard Grid

Technical Report

Submitted by: [Your Name]

Roll No: [Roll No]

Submission: Individual

Code: github.com/hariswasim617/AI-Assignment-2

1. Introduction and Problem Formulation

A disaster-response company surveys a dangerous area with a fleet of drones (UAVs). For every drone it must choose a take-off base, a route, the speed on each part of the route and when (and how strongly) the sensor is switched on. These choices conflict: covering more area costs battery and time, and the most important places are often also the most dangerous. There is no single correct answer, only trade-offs. I designed and implemented, from scratch in Python, a Genetic Algorithm (GA) [1, 2] that searches for good trade-offs; this report explains the design and the experiments.

Grid model and notation. The area is a 100×100 grid of 10 m cells (1 km × 1 km). Each cell c has *blocked(c)* ∈ {0,1} (7 no-fly zones, 8.0 % of the map), a hazard $h(c) \in [0,1]$ from 4 Gaussian wind and 3 radiation pockets, and a priority $P(c) \in [0,1]$ from 5 disaster zones. There are 30 targets (75 % inside disaster zones) and 3 base stations. Two radiation pockets are placed on purpose next to disaster zones, so priority and safety really conflict. Drone d (N = 4) flies the closed route $b_d \to w_d,1 \to \dots \to w_d,K_d \to b_d$ with range L_max = 3 km, battery B = 60 kJ and speed 3–18 m/s; its sensor radius grows from 1.5 to 5 cells with the power p ∈ [0,1] (10–60 W). The map is generated from a fixed seed, so all experiments use the same world (maps in Fig. 2).

Assumptions. Sensors look straight down (no-fly zones restrict flight, not sensing); every drone returns to its base at 14 m/s with the sensor off; propulsion power follows the rotary-wing model of Zeng et al. [4] and wind raises it by up to 60 %; detection is probabilistic and gets worse with speed (motion blur) and distance.

2. Chromosome Design and Rationale

I used a **hybrid, multi-segment, variable-length** encoding: one segment per drone, mixing an integer, real and binary genes (Table 1). The genes map one-to-one onto the planning decisions.

Table 1. Encoding of one drone segment; a chromosome has N segments and K varies per drone.

Gene (per drone)	Type	Range	Meaning
b	integer	0 … 2	launch / landing base station
x_k, y_k (k = 1…K)	real	[0, 100)	position of waypoint k (cells)
v_k	real	[3, 18] m/s	speed on the leg that ends at waypoint k
p_k	real	[0, 1]	sensor power on that leg (radius and energy)
a_k	binary	{0, 1}	sensor on/off on that leg (activation schedule)

Rationale. (1) *Real-valued coordinates*: a small gene change gives a small route change, which suits Gaussian mutation and geometric crossover (with a binary encoding one bit flip could move a drone across the map). (2) *One segment per drone*: a drone's route is a natural building block that crossover can pass on intact. (3) *Variable length* (K starts at 6–10 and may vary between 2 and 14 by mutation): the GA decides how detailed each route is. (4) *Per-leg speed, power and on/off genes* give direct control of the energy–detection trade-off, e.g. sensor off and fast on transit legs. The final plan has 58 waypoints (294 genes).

3. Fitness Function

The fitness (maximised) is a weighted sum of five objectives, each **normalised to [0, 1]**, minus a dynamic penalty:

$$F = 0.35 \cdot Cov + 0.25 \cdot Det - 0.15 \cdot E - 0.1 \cdot T - 0.15 \cdot H - \lambda(g) \cdot V$$

- **Cov** = Σ P(c)·covered(c) / Σ P(c); a cell is covered if its centre is within the sensor radius of a leg flown with the sensor on.
- **Det** = mean over targets of 1 − Π(1 − P_leg), with P_leg = q(p)·s(v)·max(0, 1 − (d/r)²), q(p) = 0.4 + 0.6p and s(v) falling linearly from 1 (v_min) to 0.5 (v_max).
- **E** = Σ E_d / (N·B), with E_d = Σ_legs [P_fly(v)·(1 + 0.6·wind) + a·P_sensor(p)]·t.
- **T** = makespan of the slowest drone / (L_max / v_min = 1000 s).
- **H** = Σ ∫h dt / Σ T_d, the time-weighted mean hazard intensity (crossing a pocket fast is better).
- **V** = 10·(share of path in no-fly zones) + range overrun + battery overrun; λ(g) = 2·(1 + 4·g/G) grows from 2 to 10 (dynamic penalty [6]).

Weights. Without normalisation the term with the largest unit (joules) would dominate and the weights would be meaningless. The two mission goals get 0.60 of the weight and the three costs 0.40, so the GA is rewarded for doing the mission rather than staying at the base. Coverage of disaster zones is the main purpose (0.35); detection (0.25) is slightly lower because good coverage already helps it. Drone safety and battery reserve are equally important (0.15 each). Time gets the smallest weight (0.1) because the range limit already bounds it and it is correlated with energy. The penalty is mild early (promising, slightly infeasible plans can help exploration) and strict at the end (the final plan must be feasible).

4. Genetic Operators and Constraint Handling

Selection: tournament (size k), linear rank (pressure 1.7) or roulette wheel; the best 5 % are copied unchanged (elitism). **Crossover** (rate 0.9): first each drone segment is taken from either parent (*segment swap*, keeps whole routes as building blocks); then, for half of the drones, coordinates, speeds and powers are blended with **BLX-α** (α = 0.3) [3], a geometric crossover that places the child between the parents, and the sensor bits use uniform crossover. **Mutation** (each gene with probability p_m): Gaussian waypoint moves with a step σ annealed from 8 to 1 cells, speed and power jitter, bit flips, base change, waypoint insert / delete / swap, and a rare (0.02) random reset of a route. **Diversity control:** the mean pairwise distance between individuals is tracked; below 30 % of its start value, 10 % random immigrants are added and p_m is doubled. **Termination:** maximum generations, stagnation (60 generations without a gain > 0.0001) or a target fitness; the criterion that fired is logged.

Repair fixes every child in four steps: (1) clip genes to their ranges; (2) move waypoints out of no-fly zones to the nearest free cell (a table built once by multi-source BFS); (3) give legs that cross a no-fly zone detour points – one side point, then divide-and-conquer, finally A∗ [5] with line-of-sight smoothing; (4) drop waypoints from the end of a route until the drone can return home

within range and battery. Repair keeps a one-cell safety margin and writes the fixed genes back (Lamarckian). Modes: *penalty* (clip + penalty only), *repair* (no penalty), *both* (default).

Table 2. Repair vs. penalty (3 seeds, 100 generations, initial settings). The best plan of every run was feasible.

Mode	Final best F (mean ± std)	Feasible share of final population	Runtime (s)
penalty	0.264 ± 0.048	54 %	18
repair	0.295 ± 0.014	100 %	90
both	0.294 ± 0.005	100 %	83

With the **penalty only**, random plans start deeply infeasible, so the GA first spends many generations becoming feasible; its final plans were worse (0.264), less consistent (± 0.048) and 46 % of its final population was still illegal. **Repair** keeps everyone feasible and gave better, more stable results (0.295 / 0.294 for *repair* / *both*) at the cost of runtime (83 s vs. 18 s), since repair (BFS, detours, A∗) is the most expensive step. *Repair* and *both* are equal because repair removes almost every violation; I kept *both* so the penalty still catches the rare case repair cannot fix [6, 7].

5. Experimental Setup

Table 3. Final configuration (p_m and k were chosen by the sensitivity analysis, Section 7).

Parameter	Value	Parameter	Value
Population size	100	Max generations	200
Crossover rate	0.9	Mutation rate p_m	0.01
Selection	tournament, k = 5	Elitism	5 %
BLX-α	0.3	Mutation σ	8 → 1 cells
Constraint mode	both	Penalty λ(g)	2 → 10
Stagnation patience	60 gen.	Seeds	main 42; sweeps 1–3; baseline 1–5

Three experiments were run: (1) the **main run** (Table 3, seed 42); (2) a **sensitivity analysis** that changes one parameter at a time around the initial settings (p_m = 0.1, k = 3), 3 seeds × 100 generations per setting with the stagnation stop off; (3) **random search** with the same random plan generator, repair, fitness (final penalty) and exactly the same number of evaluations as the GA, over 5 seeds. All runs use fixed seeds. The main run took 138 s for 19,300 evaluations (4-core cloud machine, Python 3.13 + NumPy, single process).

6. Results of the Main Run

The run stopped at the **max generations (200) reached**. The best fitness rose from 0.150 (best random plan) to **0.381** and the average from 0.016 to 0.335 (Fig. 1). Half of the gain came in the first 12 generations; afterwards the plan was refined slowly. Diversity fell below the threshold at generation 17; from then on the diversity control added immigrants in 108 of 200 generations and kept it near the threshold. These low-fitness newcomers cause the saw-tooth in the average curve, while elitism keeps the best curve monotone. 100 % of the final population was feasible.

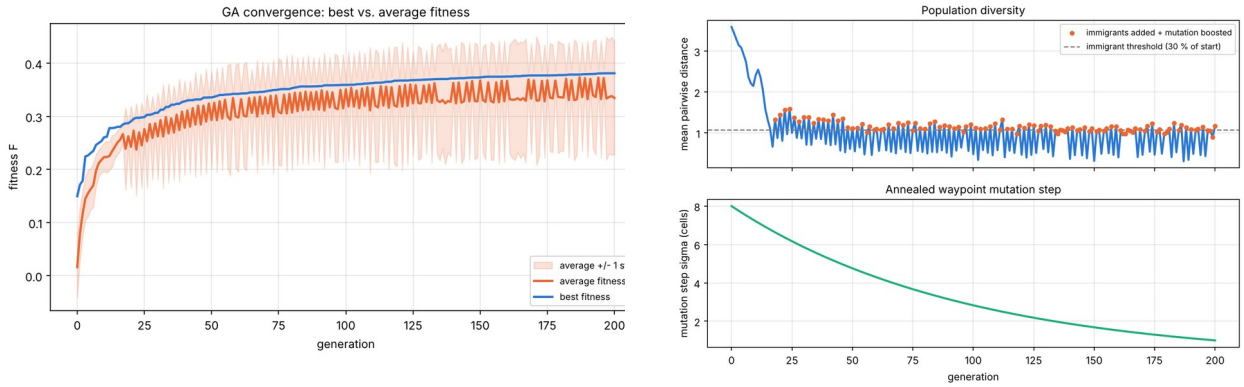


Figure 1. Left: best and average (±1 std) fitness. Right: diversity with the immigrant threshold, and the annealed mutation step σ.

Table 4. Objectives of the best plan at the start and at the end of the main run.

Best plan	Generation 0	Generation 200
Coverage / detection rate	0.550 / 0.393	0.832 / 0.869
Targets detected · high-priority cells covered	–	28 / 30 · 100 %
Energy share (kJ) / time share (makespan)	0.633 / 0.246	0.575 (138 kJ) / 0.196 (196 s)
Hazard exposure / constraint violation	0.142 / 0.000	0.143 / 0.000

The GA improved coverage, detection, energy and time together, while hazard exposure stayed almost unchanged (0.142 → 0.143): two radiation pockets lie on disaster zones, so full priority coverage requires entering them, and with these weights the GA prefers to survey them but crosses them fast. In Fig. 2, generation 0 shows long random zig-zags across the map; by generation 50 the drones concentrate on the disaster zones; at the end each drone sweeps its own area with many short legs, the sensor is off on some transit legs (dashed) and every route bends around the no-fly zones. The coverage heatmaps (Fig. 3) show that the whole population, not only the best plan, shifts its coverage from a uniform spread to the high-priority zones.

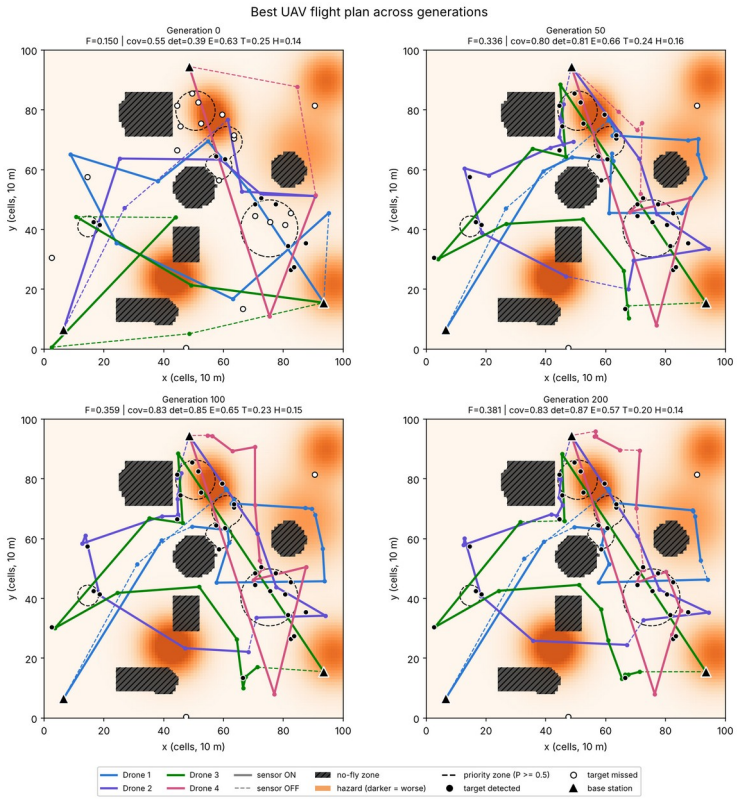


Figure 2. Best plan at generations 0, 50, 100, 200. Solid = sensor on, dashed = sensor off; grey hatched = no-fly zones; orange = hazard; dashed circles = disaster zones.

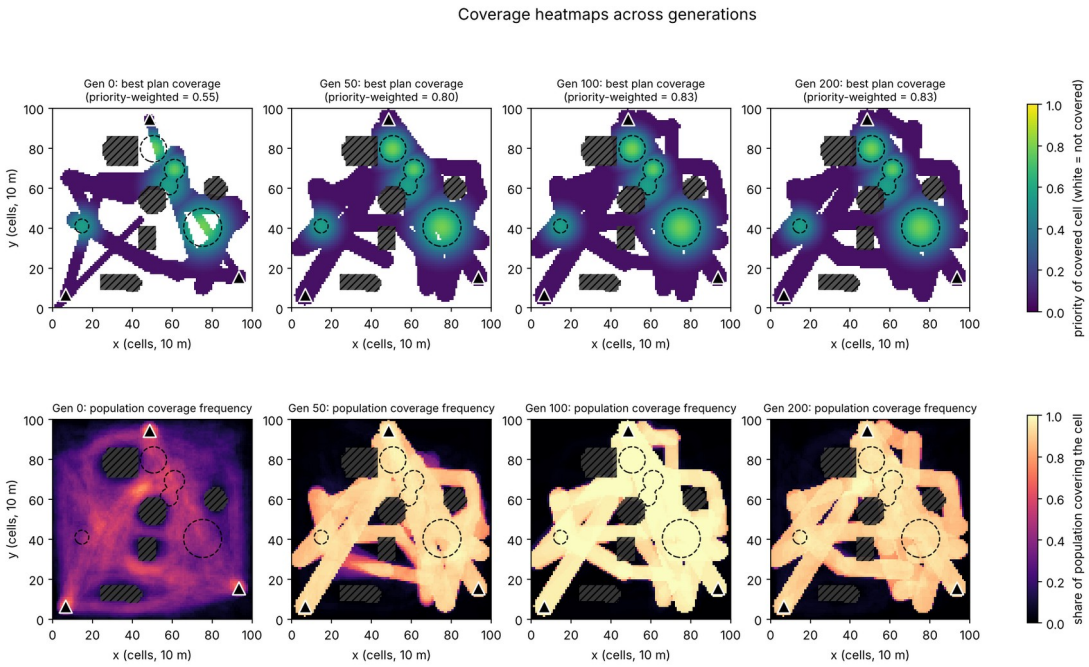


Figure 3. Top: cells covered by the best plan (colour = priority). Bottom: share of the population covering each cell.

7. Hyper-parameter Sensitivity Analysis

Each setting ran 3 seeds × 100 generations (population 100 unless swept). Table 5 gives the final best fitness (mean ± std), the generation at which 99 % of the run's improvement was reached, the final diversity, evaluations and runtime; Fig. 4 shows the mean curves.

Table 5. Sensitivity to population size N, mutation rate p_m and selection pressure (k, method).

Setting	Final best F	Gen. to 99 %	Final diversity	Evaluations	Runtime (s)
N = 20	0.280 ± 0.009	85	1.72	2,020	16
N = 50	0.293 ± 0.008	79	1.58	4,850	43
N = 100	0.294 ± 0.005	86	1.62	9,700	85
N = 200	0.320 ± 0.011	99	1.72	19,200	183
p_m = 0.01	0.356 ± 0.006	95	1.22	9,700	74
p_m = 0.05	0.326 ± 0.019	95	1.42	9,700	77
p_m = 0.1	0.294 ± 0.005	86	1.62	9,700	83
p_m = 0.2	0.273 ± 0.018	83	2.12	9,700	101
p_m = 0.4	0.262 ± 0.004	75	2.58	9,700	121
k = 2	0.295 ± 0.012	89	2.04	9,700	94
k = 3	0.294 ± 0.005	86	1.62	9,700	85
k = 5	0.326 ± 0.010	94	1.49	9,700	82
k = 8	0.325 ± 0.018	97	1.42	9,700	83
selection = roulette	0.268 ± 0.010	88	2.35	9,700	102
selection = rank	0.272 ± 0.008	92	2.43	9,700	103
selection = tournament	0.294 ± 0.005	86	1.62	9,700	83
p_m = 0.01 and k = 5	0.364 ± 0.005	–	–	9,700	67

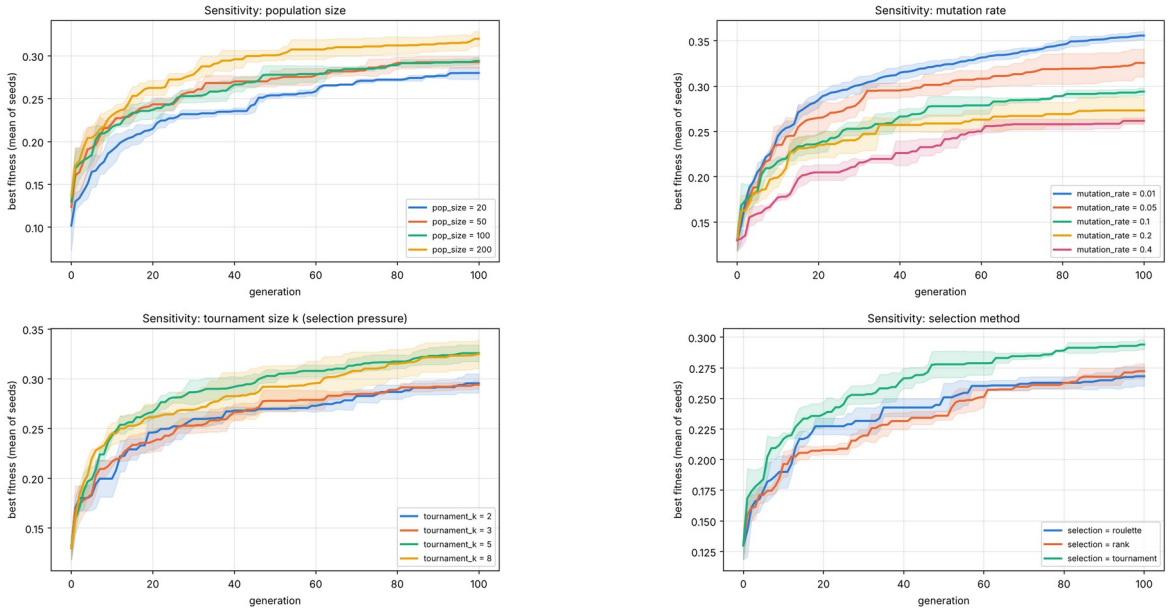


Figure 4. Mean best-fitness curves (±1 std over seeds) of the four sweeps.

Population size. Larger populations explore more of the map per generation and found better plans (0.280 → 0.320 from 20 to 200), but the cost grows linearly (16 s → 183 s) and 50 → 100 gave almost nothing (0.293 vs. 0.294), so it is a cost–quality choice.

Mutation rate had the largest effect: fitness fell steadily from 0.356 (p_m = 0.01) to 0.294 (0.1) and 0.262 (0.4), while final diversity rose from 1.22 to 2.58. A chromosome has about 150–300 genes, so p_m = 0.1 already changes some twenty genes per child: good routes are destroyed faster than selection can keep them and the search drifts towards random search (too much exploration). Damaged routes also need more repair (121 s vs. 74 s). With p_m = 0.01 the GA exploits good routes, and premature convergence is prevented by BLX-α, the annealed mutation and the immigrants instead.

Selection pressure. k = 2 and 3 performed alike (0.295, 0.294); k = 5 and 8 were clearly better (0.326, 0.325) but reduced diversity (2.04 → 1.42), and k = 8 had the largest spread between seeds (± 0.018), an early sign of premature convergence in some runs. Tournament selection (0.294) beat rank (0.272) and roulette (0.268): late in a run the fitness values are close, so roulette and rank give almost everyone a similar chance and the search drifts (they keep the highest diversity, 2.35 and 2.43).

Final setting. Because the sweeps vary one parameter at a time, I also ran the two best values together (p_m = 0.01, k = 5): 0.364 ± 0.005, better than either alone, with immigrants active in 39–46 of 100 generations, so diversity stayed under control. This is the final configuration. Most runs were still improving after 100 generations (99 % of the gain only after generation 75–99), so the main run uses 200.

8. GA versus Random Search

Table 6. GA vs. random search: mean ± std over 5 seeds, 19,300 evaluations each. All best plans were feasible (violation 0).

Best plan	Genetic Algorithm	Random search
Fitness F	0.383 ± 0.010	0.208 ± 0.009
Coverage (priority-weighted)	0.831 ± 0.013	0.670 ± 0.039
High-priority cells covered	0.999 ± 0.001	0.893 ± 0.066
Detection rate	0.856 ± 0.028	0.492 ± 0.042
Targets detected (of 30)	28.2 ± 0.8	16.2 ± 2.4
Energy (kJ)	134.2 ± 4.0	157.0 ± 15.9
Makespan (s)	199.9 ± 14.0	261.4 ± 22.6
Hazard exposure	0.123 ± 0.008	0.169 ± 0.033

The GA was better on **every** metric and in every seed: its worst seed (0.373) is far above the best random seed (0.224). In the main run it passed the final random-search level (0.208) after only 388 evaluations, while random search improves only very slowly after its first thousand samples (Fig. 5). The GA was even faster (135 s vs. 371 s per run) because its children need little repair, whereas every random plan needs heavy repair.

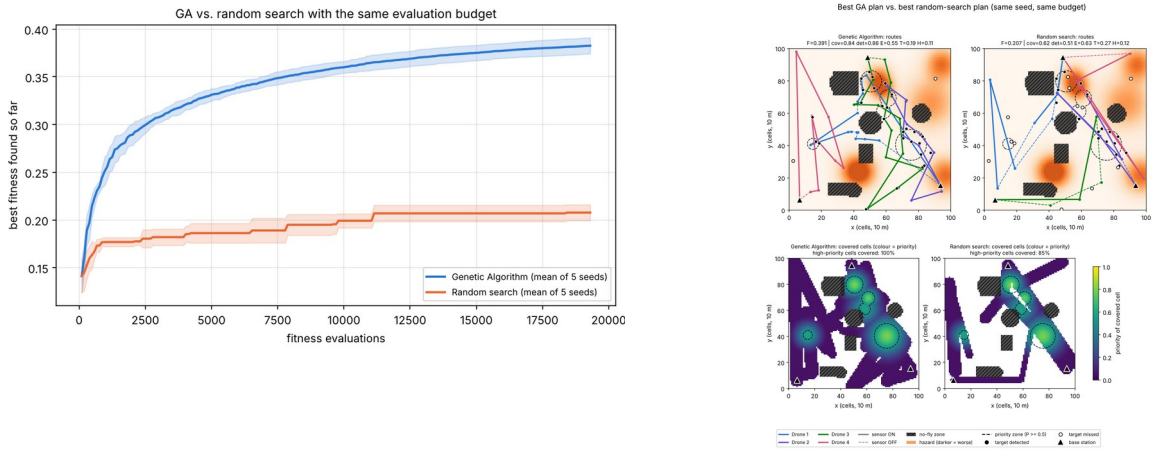


Figure 5. Left: best-so-far fitness vs. evaluations (mean ± std). Right: best plans of seed 1, routes (top) and covered cells (bottom).

What makes the evolved solution different? (i) *Structured paths:* random plans are a few long straight legs across the map that hit the range limit and are cut short by repair (17 waypoints in seed 1); the GA builds many short legs (46 waypoints) that sweep the disaster zones, and the drones share the map. (ii) *Sensor scheduling:* the GA keeps the sensor on for 89 % of the legs at high power (mean 0.86) and switches it off on a few low-priority legs (random: 76 %, power 0.67); it pays for this by flying faster and finishing sooner, so it still uses less energy. (iii) *Priority focus:* 99.9 % of high-priority cells covered (random 89.3 %) and 28.2 instead of 16.2 targets found. (iv) *Hazard avoidance:* it must enter the radiation pockets on the disaster zones but crosses them quickly and mostly avoids the others (exposure 0.123 vs. 0.169). Random search finds such a coordinated plan only by chance; the GA builds it by keeping and recombining good drone routes.

9. Limitations and Future Work

(1) A weighted sum gives one compromise per weight vector; a Pareto method (e.g. NSGA-II) would offer a set of trade-offs. (2) The world is static and simplified (fixed hazards, no wind drift, no drone collisions, simple sensor footprint) and only one map was used. (3) Repair, especially A∗, dominates the runtime. (4) Tuning was one-factor-at-a-time with three seeds; a factorial design with more seeds would give stronger evidence.

10. Conclusion

I built a GA that plans the deployment, routes, speeds and sensor schedules of a 4-drone fleet on a 100×100 hazard grid. The hybrid multi-segment chromosome maps directly onto the decisions, the normalised weighted fitness balances five conflicting objectives, and repair plus a dynamic penalty keeps every final plan feasible. The sensitivity study showed that this problem rewards exploitation (low mutation rate, strong tournament) as long as diversity is controlled. With the tuned settings the GA reached 0.383 ± 0.010 against 0.208 ± 0.009 for random search with the same budget, with better values on every objective. A problem-specific encoding and operators let a simple GA [1, 2, 8] clearly outperform undirected search.

References

[1] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Univ. of Michigan Press, 1975.
[2] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
[3] L. J. Eshelman, J. D. Schaffer, "Real-coded genetic algorithms and interval-schemata," *Foundations of Genetic Algorithms 2*, pp. 187–202, 1993.
[4] Y. Zeng, J. Xu, R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, 18(4), pp. 2329–2345, 2019.
[5] P. E. Hart, N. J. Nilsson, B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, 4(2), pp. 100–107, 1968.
[6] J. A. Joines, C. R. Houck, "On the use of non-stationary penalty functions to solve nonlinear constrained optimization problems with GAs," *Proc. IEEE CEC*, pp. 579–584, 1994.
[7] C. A. Coello Coello, "Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey," *Comput. Methods Appl. Mech. Eng.*, 191, pp. 1245–1287, 2002.
[8] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.